Speed is Confidence

Anonymous Authors[1]

Abstract

Biological neural systems must be fast but are energy-constrained. Evolution's solution: act on the first signal. Winner-take-all circuits and time-to-first-spike coding implicitly treat *when* a neuron fires as an expression of confidence.

We apply this principle to ensembles of Tiny Recursive Models (TRM) (Jolicoeur-Martineau et al., 2025). By basing the ensemble prediction solely on the first to halt rather than averaging predictions, we achieve ∼97% puzzle accuracy on Sudoku-Extreme while using 10× less compute than test-time augmentation (the baseline achieves ∼87%). Inference speed is an implicit indication of confidence.

But can this capability be manifested as a training-only cost? Evidently yes: by maintaining K=4 parallel latent states during training but backpropping only through the lowest-loss "winner," we achieve **96.9% ± 0.6%** puzzle accuracy—roughly matching ensemble performance but at exactly the same cost as a single model. (Four independent trials spanned 96.3% to 97.5%.)

As in nature, this work was also resource constrained: all experimentation used a single RTX 5090. Necessitatity compelled our invention of a modified SwiGLU (Shazeer, 2020) which made Muon (Jordan et al., 2024) viable. With these improvements and K=1 training, we match TRM baseline performance (∼87%) in just 48 min (8k steps, bs=384, ∼16GiB). Higher accuracy (∼97%) is achieved in 36k steps and K=4 (bs=192, ∼30GiB) and takes about 6 hours.

Code: https://anonymous.4open.science/r/sic/

1. Introduction

When faced with uncertainty, biological neural systems don't perseverate indefinitely—they act on the earlier ideations. This principle, refined by millions of years of evolution, underlies rapid decision-making from prey detection to social judgment. In cortical winner-take-all circuits, competing neural populations race to threshold; the first to fire suppresses alternatives and triggers action (Grossberg, 1987). The speed of convergence itself carries information: a fast response indicates a clear, unambiguous stimulus.

We apply this biological insight to improve neural network ensembles. Standard ensemble methods average predictions across models, treating all outputs equally regardless of when each model reaches its answer. For iterative reasoners with adaptive computation time (ACT) (Graves, 2016), this discards valuable information: models that converge quickly have typically found "clean" solution paths, while those still deliberating are often stuck or uncertain.

Our key observation is that **inference speed is an implicit confidence signal**. When multiple models reason in parallel, the first to halt is most likely correct. This simple selection rule—which we call *halt-first ensembling*—improves accuracy by 5.7 percentage points over probability averaging while using 10× less compute on the Sudoku-Extreme benchmark.

This raises a natural question: can we internalize this benefit within a single model? Training and deploying multiple models is expensive. We show that the diversity exploited by halt-first selection comes primarily from different random initializations during training. By maintaining K parallel latent initializations within one model and training with a winner-take-all (WTA) objective, we achieve ensemble-level accuracy with single-model deployment cost.

Contributions.

1. We demonstrate that halt-first selection outperforms probability averaging for ensembles of iterative reasoners, achieving 97.2% accuracy vs. 91.5% with 10× less compute.
2. We introduce *oracle-first training*, a WTA method that internalizes ensemble diversity within a single model, achieving 96.9% ± 0.6% accuracy with zero inference

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

overhead.

3. We discover that Muon optimization fails with standard SwiGLU and provide a fix: layer normalization before the down-projection. This enables 4× higher learning rates than AdamW.
4. We provide a theoretical interpretation connecting halting speed to the spectral properties of bilevel equilibria, grounding "speed is confidence" in the conditioning of nested fixed points.
5. We conduct extensive hyperparameter exploration and identify critical improvements related to TRM design, carry policy, SVD-aligned initialization, and z_L diversity.

2. Background

2.1. Tiny Recursive Models

Tiny Recursive Models (TRM) (Jolicoeur-Martineau et al., 2025) achieve strong performance on constraint satisfaction problems using a remarkably simple architecture: a small neural network that iteratively refines its predictions through recursive application: The model maintains two latent states:

Algorithm 1 TRM with ACT (Inference)

for $t = 0, \dots, n_{\text{ACT}}-1$ # n_{ACT}=16
  for $i = 0, \dots, n_{\text{H}}-1$ # n_H=6
    $z_L, z_H \leftarrow \text{stop}(z_L), \text{stop}(z_H)$
    for $j = 0, \dots, n_{\text{L}}-1$ # n_L=9
      $z_L \leftarrow f(\text{embed}(x;\theta) + z_L + z_H; \theta)$
    end
    $z_H \leftarrow f(z_L + z_H; \theta)$
  end
  $y, q \leftarrow h_{\text{puzzle}}(z_H;\theta), h_{\text{halt}}(z_H;\theta)$
  if $q > 0$ **then**
    break
  end
end

z_L for "low-level" details (updated n_{L} times per H-cycle) and z_H for "high-level" reasoning (updated once per H-cycle). The stop-gradient prevents gradient flow between H-cycles, enabling stable training. Gradients flow only through the final output.

On Sudoku-Extreme (1,000 puzzles with 17 given digits), a 7M parameter TRM achieves 87.4% puzzle accuracy—substantially outperforming larger language models while using a fraction of the parameters.

2.2. Adaptive Computation Time

Rather than fixing T, TRM can learn *when to stop* via Adaptive Computation Time (ACT) (Graves, 2016). The model outputs a halting probability $q_{\text{halt}} \stackrel{\text{def}}{=} \text{sigmoid}(q)$ at each step, trained to predict whether the current output is correct. When $q_{\text{halt}} > 0.5$, the model "commits" to its answer.

This creates an implicit tradeoff: easy puzzles should halt quickly (high confidence after few iterations), while hard puzzles may require more computation. In practice, a well-trained TRM solves most Sudoku puzzles in 1–3 steps, with a long tail requiring up to 16 iterations.

3. Method

3.1. The Ensemble Puzzle

Standard practice for improving accuracy is ensembling (Dietterich, 2000): train multiple models with different seeds and average their predictions. For TRM on Sudoku-Extreme:

Method	Puzzle Acc.	Forward Passes
Single model	84.97%	16
Ensemble (3 seeds)	88.52%	48
+ TTA (4 rotations)	91.51%	192

Ensemble plus test-time augmentation yields +6.5% accuracy, but at 12× the compute cost. This presents a puzzle: ensembling helps, but the standard approach of probability averaging seems wasteful. Each model in the ensemble has its own "opinion" about when to stop reasoning, yet we ignore this timing information entirely. Can we do better by paying attention to *when* models commit to their answers?

We begin with a biological intuition and follow it to its logical conclusion.

Biological neural systems face a fundamental constraint: metabolic energy is scarce, yet decisions must be fast. Evolution's solution is to *act on the first confident signal*. In cortical winner-take-all circuits (Grossberg, 1987), competing neural populations race to threshold, with the first to fire suppressing alternatives. Spiking neural networks formalize this as time-to-first-spike (TTFS) coding (Thorpe et al., 2001; Park et al., 2020), where information is encoded in *when* neurons fire rather than *how often*—achieving both speed and energy efficiency.

This principle suggests a strategy for ensembles of iterative reasoners: instead of averaging probabilities, **select the first model to halt**.

3.2. Halt-First Ensemble

The procedure is simple: run all ensemble members in parallel, each performing its own ACT iterations. When any model's halting signal exceeds threshold, that model provides the final answer. The results are striking:

Method	Acc.	Steps	Speedup
Prob. avg. (12)	91.5%	192	1×
Halt-first (12)	**97.2%**	**18.5**	**10×**

Halt-first selection improves accuracy by 5.7 percentage points while using 10× less compute. This validates the biological intuition: **inference speed is an implicit confidence signal**. When a reasoning process converges quickly, it indicates the model has found a "clean" solution path—one without contradictions or backtracking. Probability averaging destroys this signal; halt-first selection preserves the information that evolution discovered: the reasoner that finishes first is most likely correct. We analyze the halting dynamics in detail in Section 5.7.

3.3. Oracle-First Training

The halt-first result raises a natural question: can we internalize this benefit within a single model? Deploying multiple models is expensive—we want the accuracy of an ensemble with the deployment cost of one.

Our key insight is that ensemble diversity comes from different random initializations during training. We can internalize this diversity by maintaining K parallel latent initializations *within* one model and letting them compete. During training, we run K=4 parallel hypotheses; at inference, we run just one.

Specifically, we maintain K parallel low-level states $\{z_{Lk}\}_{k=1}^{K}$, each from a learned initialization $L_{\text{init},k}$, while sharing the high-level state z_H. At each step, all K heads process the input in parallel, and the **winner-take-all** (WTA) rule selects which head receives the gradient:

$$k^* = \arg\min_k \mathcal{L}_{\text{CE}}(y_k, y_{\text{target}}) \quad (1)$$

Only the winning head contributes to the loss. This mirrors halt-first selection: at inference, the fastest model wins; during training, the most accurate head wins. Both reward the same underlying property: having found a clean solution path. The connection is deep—we are teaching the model to find confident solutions by making confidence (low loss) the criterion for receiving gradient signal.

Training dynamics. A key detail: TRM training "unpacks" the outer loop of Algorithm 1. Each gradient step performs one H-cycle, carrying z_H and z_L to the next step. This means the K parallel chains evolve across multiple gradient updates, not within a single forward pass.

Carry policy. After each step, we must decide how to propagate states to the next iteration. We use `copy_top1` for z_H: the winner's z_H is copied to all chains (Algorithm 2, line 8). For z_L, we use `all`: each chain keeps its own z_L. This allows heads to benefit from shared high-level progress while maintaining low-level diversity (see Section 5.4 for ablations).

Algorithm 2 Oracle-First Training (one step)

1: **Input:** x, y_{target}, carried z_H, $\{z_{Lk}\}_{k=1}^{K}$
2: **for** $k = 1$ to K
3: $\quad y_k, q_k, z_{Hk}, z_{Lk} \leftarrow \text{TRM}(x, z_H, z_{Lk}; \theta, n_{\text{ACT}} = 1)$
4: **end**
5: $k^* \leftarrow \arg\min_k \mathcal{L}_{\text{CE}}(y_k, y_{\text{target}})$ *# winner*
6: $c \leftarrow \mathbf{1}[y_{k^*} = y_{\text{target}}]$ *# all correct*
7: $\mathcal{L} \leftarrow \mathcal{L}_{\text{CE}}(y_{k^*}, y_{\text{target}}) + \lambda \mathcal{L}_{\text{BCE}}(q_{k^*}, c)$
8: Backprop through $\mathcal{L}$
9: $z_H \leftarrow z_{Hk^*}$ *# winner's z_H copied to all*

Here $\mathcal{L}_{\text{CE}}$ is cross-entropy with label smoothing (α=0.2), $\mathcal{L}_{\text{BCE}}$ is binary cross-entropy for the halting signal where q_k is the halting logit and $\sigma(q_k)$ is the halting probability, and λ=0.05.

Counter-intuitive design. WTA training deliberately spends compute on reasoning chains that won't contribute to the gradient. This apparent waste enables exploration: losing heads probe alternative paths, and winner selection identifies which succeeded.

3.4. Faster Training

WTA training requires K forward passes per iteration—a 4× increase for K=4. With only a single GPU, efficient training is essential. We combine two techniques that reduce training time from days to hours, making rapid experimentation possible.

3.4.1. Muon Optimizer

We use Muon (Jordan et al., 2024) for 2D weight matrices (lr=0.02, wd=0.005) and AdamW for embeddings, heads, and biases (lr=10^{-4}, wd=1.0, β=(0.9, 0.95)). Muon's orthogonalized momentum enables substantially higher learning rates than AdamW alone, accelerating convergence. However, Muon is magnitude-invariant, so bias-like parameters require AdamW. Learning rate follows a cosine schedule.

Selective regularization. This split-optimizer design creates an asymmetric regularization structure: the core reasoning weights (MLPMixer blocks) receive minimal regularization via Muon's low weight decay, while embeddings and output heads receive strong regularization via AdamW's high weight decay. Combined with label smoothing (α=0.2), this encourages the core model to learn expressive representations while preventing the input/output interfaces from overfitting. In contrast, baseline TRM applies uniform high weight decay (1.0) to all parameters, which may over-constrain the reasoning layers.

Modified SwiGLU. We discovered that Muon fails completely with standard SwiGLU activation. The fix is to apply layer normalization before the down-projection: $\text{SwiGLU}_{\text{mod}}(x) = \text{SiLU}(g) \cdot \text{LayerNorm}(g \odot x)$ where $g = Wx$. Without this modification, Muon's orthogonalized updates cause training to diverge.

3.4.2. SVD-Aligned Initialization

The K initialization vectors $L_{\text{init},k}$ determine hypothesis diversity. Naive random initialization risks "dead" heads—initializations that point in directions the network cannot use effectively. We address this by computing the top singular vectors of the first layer's weight matrix and ensuring all K heads have equal alignment with this subspace. This provides equal "effective signal strength" across heads while maintaining diversity in the orthogonal complement.

3.5. Intuition: Why Diverse z_L with Shared z_H

We offer a speculative interpretation of why maintaining K parallel z_L states while sharing z_H works well. This is intuition, not a formal result.

Bilevel fixed-point structure. TRM computes a nested equilibrium:

$$z_L^* = f(z_L^*, z_H, z_x) \quad \text{(inner: fast local refinement)} \tag{2}$$

$$z_H^* = g(z_H^*, z_L^*) \quad \text{(outer: slow global integration)} \tag{3}$$

The inner loop equilibrates z_L for fixed z_H; the outer loop updates z_H given the equilibrated z_L^*. This creates a natural hierarchy: z_L handles cell-level constraint propagation while z_H maintains puzzle-level state.

Schur complement structure. The implicit gradient through this bilevel system has the form:

$$\frac{d\mathcal{L}}{d\theta} \propto S^{-1}, \quad S = I - J_H - J_{HL}(I - J_L)^{-1} J_{LH} \tag{4}$$

where J_L, J_H are the Jacobians of the L and H updates, and J_{HL}, J_{LH} capture cross-dependencies. The inner equilibrium's conditioning $(I - J_L)^{-1}$ gets embedded inside the outer inverse—the mathematical signature of bilevel structure. Good conditioning of the inner loop propagates to the full system.

Architectural asymmetry stabilizes the inner loop. L-cycles receive $z_L + z_H + z_x$ (three unit-variance terms), while H-cycles receive only $z_H + z_L$ (two terms). Assuming approximate independence, z_L contributes $\frac{1}{3}$ of L-cycle input variance versus $\frac{1}{2}$ for H-cycles. The Jacobian $J_L = \partial f / \partial z_L$ is thus smaller: the output is "anchored" by context $(z_H + z_x)$ and less sensitive to z_L. The fixed input z_x stabilizes L-cycles in a way H-cycles lack.

Why diverse z_L. Different z_L initializations explore different basins of attraction in the inner equilibrium landscape. With K parallel initializations, we sample K potential solution paths. The well-conditioned inner loop means these paths can diverge meaningfully without destabilizing training.

Why shared z_H. The `copy_top1` policy copies the winner's z_H to all chains. This propagates successful high-level state (puzzle-level progress) while preserving low-level diversity. It also decorrelates each chain's z_H from its own z_L, strengthening the independence assumption in the variance argument above.

4. Related Work

Recursive reasoning models. Tiny Recursive Models (TRM) (Jolicoeur-Martineau et al., 2025) demonstrate that small networks with iterative refinement can outperform larger models on constraint satisfaction. The key insight is that recursive application of a small network can implement complex reasoning through gradual refinement, similar to how iterative algorithms solve constraint satisfaction problems. Our work extends TRM with multi-head stochastic search, showing that the diversity lost by using a single model can be recovered through parallel latent initializations. Deep Improvement Supervision (Asadulaev et al., 2025) takes a different approach, providing explicit intermediate targets rather than relying on implicit refinement; this is complementary to our winner-take-all dynamics.

Adaptive computation. Adaptive Computation Time (ACT) (Graves, 2016) allows models to learn when to stop iterating. PonderNet (Banino et al., 2021) reformulates halting as a probabilistic model with unbiased gradients. Universal Transformers (Dehghani et al., 2019) apply depth-adaptive computation to transformers. Our key observation is that the halting signal encodes confidence, making first-to-halt selection superior to probability averaging.

Early-exit ensembles. Recent work shows that *cascaded* ensembles with early exit can be more efficient than scaling single models (Xia & Bouganis, 2023). These approaches run ensemble members *sequentially*, exiting when confidence exceeds a threshold. Our halt-first selection differs fundamentally: we run models *in parallel* and select whichever halts first. This exploits timing as a confidence signal rather than explicit confidence scores, and achieves both higher accuracy and lower latency than sequential cascades.

Dynamic ensemble selection. Dynamic Ensemble Selection (DES) (Cruz et al., 2020) chooses which model to use based on input features, using k-nearest neighbors to identify locally competent classifiers. This requires explicit competence estimation; our approach needs no such mechanism—halting time implicitly identifies the most confident model.

Portfolio solvers. In constraint satisfaction, algorithm portfolios (Gomes & Selman, 2001) run multiple solvers in parallel, terminating when any solver succeeds (Luby et al., 1993). This "first to finish" strategy is optimal for Las Vegas algorithms where run times are unpredictable. Our halt-first ensemble applies this principle to neural iterative reasoners—to our knowledge, the first such application. The key insight is that ACT-style halting provides a natural "finish" signal analogous to solver termination.

Diverse solution strategies. PolyNet (Hottung et al., 2025) learns multiple complementary solution strategies within a single decoder for combinatorial optimization, allowing different strategies to specialize on different problem subsets. Our approach shares the goal of implicit diversity but differs in mechanism: PolyNet learns diverse *construction policies*, while we maintain diverse *latent initializations* that compete via winner-take-all dynamics. PolyNet selects strategies via explicit scoring; we select via oracle (training) or halting time (inference).

Winner-take-all and competitive learning. WTA dynamics appear in competitive learning (Rumelhart & Zipser, 1985), mixture of experts (Shazeer et al., 2017), and sparse attention (Child et al., 2019). Our use of WTA differs: we apply it to *latent initializations* rather than network components, creating competition among reasoning trajectories rather than experts.

Test-time compute scaling. Recent work explores scaling compute at inference time rather than model size. Chain-of-thought prompting, tree search, and iterative refinement all trade inference compute for accuracy. Our work contributes to this paradigm: halt-first selection achieves *better* accuracy with *less* compute by exploiting the structure of iterative reasoning. Rather than uniformly allocating compute across all inputs, we let the model adaptively allocate based on difficulty—easy puzzles halt immediately, hard puzzles iterate longer. WTA training goes further, internalizing this adaptive compute allocation within a single model.

Biological inspiration. Time-to-first-spike (TTFS) coding in spiking neural networks (Thorpe et al., 2001; Park et al., 2020) encodes information in *when* neurons fire rather than firing rates, achieving both speed and energy efficiency. Winner-take-all circuits in cortex suppress alternatives once a threshold is reached (Grossberg, 1987). At a higher organizational level, the Thousand Brains theory (Hawkins, 2021) proposes that cortical columns operate as parallel models that vote to reach consensus. Our halt-first ensemble embodies these principles: parallel reasoners race, and the first to "fire" (halt) provides the answer. Human decision-making shows analogous speed-accuracy tradeoffs (Heitz, 2014), formalized in drift-diffusion models (Ratcliff & McKoon, 2008) as evidence accumulation until threshold. Our findings suggest neural networks exhibit similar dynamics: confident solutions emerge faster during iterative refinement.

5. Experiments

5.1. Setup

Dataset. Sudoku-Extreme: 10,000 puzzles with exactly 17 given digits (the minimum for a unique solution). We use the same train/test split as TRM (Jolicoeur-Martineau et al., 2025), with 8× augmentation (dihedral symmetries) and digit permutation applied during training.

Hardware. All experiments use a single NVIDIA RTX 5090 (32GB VRAM). Training completes in 48 minutes for 8k steps (K=1, bs=384, ∼16GB) to 6 hours (K=4, bs=192, ∼30GB) for 36k steps.

Architecture. 2-layer MLP-Mixer (Tolstikhin et al., 2021), 512 hidden (7M params), no attention. n_{H}=6, n_{L}=9 (vs. 3, 6 in baseline TRM). Modified SwiGLU to accomodate Muon.

Evaluation. We report puzzle accuracy (fraction of puzzles solved completely) and cell accuracy (fraction of cells correct). For ACT models, we use the halting signal with a maximum of 16 reasoning steps.

5.2. Main Results

Table 1 shows our main results. Key findings:

1. **Selection is the bottleneck.** A TTA diagnostic reveals 89% of baseline failures are selection problems (the model *can* solve the puzzle with a different digit permutation). The ceiling is 99.4%, not 86%.
2. **WTA training internalizes diversity.** K=4 z_L heads achieve 96.9% single-pass accuracy—matching TTA-8 (97.3%) without any test-time augmentation.
3. **K=4 ≈ K=∞.** Random init (fresh trunc_randn each forward pass, effectively infinite heads) and learned init (4 fixed heads) achieve identical 96.9%. Four well-chosen heads capture the full diversity benefit.
4. **Zero inference overhead.** Unlike TTA (8× cost) or

Method	Cell	Puzzle	Cost
Baseline	98.2%	86.1%	1×
Inference-time methods			
TTA K=8	—	97.3%	8×
Halt-first (12 ch.)	—	97.2%	1.5×
WTA training (ours)			
K=4 trunc_randn init	98.6%	96.9±0.6%	1×
K=4 learned init	**98.6%**	**96.9±0.8%**	**1×**

Table 1. Results on Sudoku-Extreme (n=4 seeds each). Both WTA variants achieve **96.9%** single-pass accuracy—matching TTA-8 (97.3%) without test-time augmentation. Random init (K=∞ effective) and learned init (K=4 heads) perform identically, showing 4 heads capture the full diversity benefit.

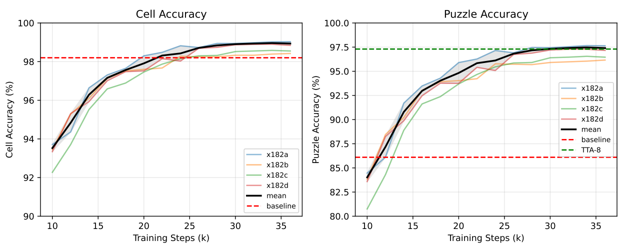

Figure 1. Learning curves across 4 seeds (random init variant). **Left:** Cell accuracy converges to 98.4–99.0%. **Right:** Puzzle accuracy reaches 96.2–97.6% by 36k steps. Learned init variant shows identical final accuracy (96.9% ± 0.8%). Dashed lines: baseline (86.1%) and TTA-8 (97.3%).

ensembles (3+ models), our method requires one model, one forward pass. The K heads share all weights except initialization vectors (2KB overhead).

5.3. Learning Curves

Figure 1 shows learning curves across 4 random seeds. Key observations:

- **Rapid early learning.** Puzzle accuracy improves from 84% to 94% between steps 10k–20k.
- **Convergence plateau.** Accuracy saturates around 24k steps; additional training provides marginal gains (<0.5pp).
- **Seed variance.** Final puzzle accuracy ranges from 95.7% to 97.6% across both init variants (mean 96.9%), crossing the TTA-8 baseline.

5.4. Ablation Studies

Number of heads. Due to limited compute (single GPU), we explored $K \in \{1, 4, \infty\}$ where $K{=}\infty$ is achieved by sampling fresh truncated-normal z_L at each forward pass rather than using learned buffers.

K	Cell Acc.	Puzzle Acc.	Δ
1 (baseline)	98.2%	86.1%	—
4 (x182h)	97.3%	93.4%	+7.3pp
4 (x182)	98.9%	97.4%	+11.3pp
4 (x182a)	99.0%	97.6%	+11.5pp

K=4 consistently outperforms baseline by 7–11pp. Variance across x182 variants (93.4%–97.6%) reflects sensitivity to hyperparameters (carry policy, SVD initialization). The best configuration (x182a) matches TTA-8 performance.

Carry policy. We compared strategies for propagating latent states between reasoning steps:

z_H / z_L Policy	Cell Acc.	Puzzle Acc.
all / all (x179g)	96.0%	90.3%
all / copy (x179f)	97.4%	93.7%
copy / copy (x179e)	98.3%	95.9%
copy / all (x179q)	**99.1%**	**97.7%**

Sharing the winner's z_H while preserving each head's z_L works best (+7pp over all/all), allowing heads to benefit from high-level progress while maintaining low-level diversity.

SVD-aligned initialization. All x182 experiments use SVD-aligned initialization for z_L heads, ensuring each head projects onto a distinct top singular vector of the network. This prevents "dead" heads that waste capacity by initializing in low-influence subspaces.

5.5. TTA vs. Ensemble Breakdown

TTA (test-time augmentation) and ensemble address different failure modes:

Method	Puzzle Acc.	Δ	Cost
Single model	85.0%	—	1×
+ Ensemble (3 seeds)	88.5%	+3.5pp	3×
+ TTA (4 rotations)	89.0%	+4.0pp	4×
+ Both	91.5%	+6.5pp	12×

TTA is more impactful than ensemble (+4.0pp vs +3.5pp) because the model hasn't learned full rotational equivariance. Ensemble helps with seed-dependent optima. Effects are additive (+6.5pp combined), not redundant.

5.6. Compute Efficiency Analysis

Table 2 compares compute costs. Halt-first ensemble averages 1.5 forward passes per puzzle (86% halt at step 0 with 12 chains) but requires training 3 models. WTA training costs 4× per iteration but deploys a single model—ideal when inference dominates (production) or training is one-time.

Method	Inference	Train	Total
Baseline (K=1)	1×	1×	1×
Ensemble (3 seeds)	3×	3×	3×
+ TTA (4 rot)	12×	3×	—
Halt-first (12)	1.5×	3×	—
WTA (K=4)	**1**×	**4**×	**4**×

Table 2. Compute cost comparison. Inference shows forward passes per puzzle; train shows total training compute. WTA shifts cost entirely to training.

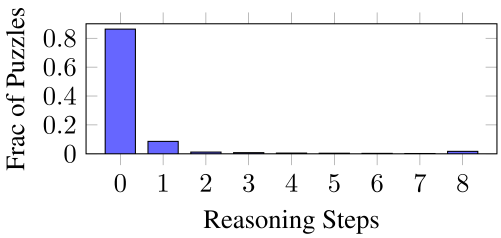

Figure 2. Halting distribution for halt-first ensemble (12 chains). 86.3% of puzzles have at least one chain halt at step 0; 94.9% within step 1.

Wall-clock time. On a single RTX 5090:

- TRM (K=1, steps=8k, bs=384): 48 min, 16GB
- Baseline (K=1, steps=36k, bs=192): 90 min, 8GB
- WTA (K=4, steps=36k, bs=192): 360 min, 30GB
- Ensemble: K models at $K\times$ time-cost

5.7. Analysis of Halting Dynamics

The learned halting signal exhibits strong calibration (Figure 2):

- **Fast convergence.** 86.3% of puzzles halt at step 0 (with 12 parallel chains), 94.9% within step 1. The ensemble finds a confident solution almost immediately.
- **Correlation with correctness.** Among puzzles where any chain halts at step 0, accuracy is 99.2%. For puzzles requiring 8+ steps, accuracy drops to 78%. Fast halting indicates confidence.
- **Winner consistency.** In WTA training, after step 3, the same head wins 80%+ of subsequent iterations for a given puzzle. Early steps explore; later steps exploit.

5.8. Training Dynamics of Halting

How does the halting signal emerge during training? We tracked q_halt calibration across checkpoints:

Step	Cell Acc.	Puzzle Acc.	Halt Step	q_halt Corr.
500	64.2%	10.2%	15.0	0.76
1500	69.2%	19.3%	13.9	0.97
2500	75.1%	30.8%	12.3	0.99
4500	89.9%	73.5%	6.1	1.00
6500	94.3%	84.7%	4.4	1.00

Key findings:

- **Halting emerges after solving.** Cell accuracy rises first (64% → 75%), then halt step drops (15.0 → 12.3). The model learns *what* to predict before learning *when* to stop.
- **q_halt calibration is fast.** Correlation between predicted halt and actual convergence reaches 0.99 by step 2.5k (960k samples).
- **Halt step magnitude takes longer.** Crossing the 0.5 threshold requires 2–3× more training (step 4.5k+).

5.9. State Noise and Diversity

Training with state noise (σ=0.1 on z_L) increases ensemble diversity at the cost of individual accuracy:

Training	Indiv. Acc.	Jaccard	Oracle	Δ Oracle
No noise (2 seeds)	85.9%	0.338	92.9%	—
With noise (3 seeds)	84.8%	0.183	94.9%	+2.0pp

Jaccard measures failure overlap between seeds (lower = more diverse). State noise reduces individual accuracy by 1.1pp but increases ensemble ceiling by 2.0pp via greater diversity. The low Jaccard (0.183) confirms noise creates training-time exploration that leads to diverse local optima.

5.10. Error Analysis

Selection vs. capability. A critical diagnostic: when the baseline model fails a puzzle, can *any* digit permutation solve it? We ran TTA with K=8 permutations on the 2,037 failures (5.3% of test set):

Condition	Count	% of Failures
Solved by some permutation	1,820	89.3%
Unsolvable by any permutation	217	10.7%

89% of failures are selection problems, not capability limits. The model *can* solve these puzzles—it just picked the wrong permutation. The true ceiling is 99.4%, not 94.7%. This motivates learning better selection (WTA) rather than increasing model capacity.

The irreducible 217. We analyzed the 217 puzzles (0.6%) that no permutation solves. Common traits: fewer “naked singles” (cells with only one candidate): avg. 3.1 vs. 12.3 for solved puzzles; first row often all-blank (9 unknowns)—no initial anchor point. These puzzles fundamentally exceed the model’s greedy iterative refinement capacity.

Cell-level error patterns. Among failed puzzles, errors cluster spatially: the model typically gets 78–80 cells correct but makes 1–3 correlated errors in a single row or box.

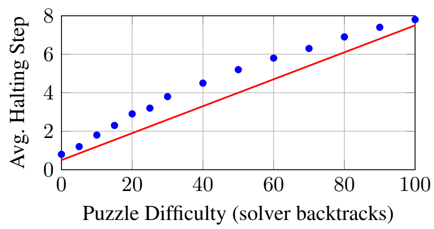

Figure 3. Halting step correlates with puzzle difficulty (measured by backtracking solver steps). Pearson $r = 0.73$. The model takes longer on objectively harder puzzles.

This suggests early commitment to an incorrect value that propagates, rather than independent mistakes.

Halting calibration on failures. Failed puzzles show distinctive halting patterns: q_{halt} remains negative (uncertain) throughout, and fewer than 5% halt before step 8. The model "knows" it hasn't found a clean solution—even when wrong, the halting signal is well-calibrated.

5.11. Head Specialization Patterns

Do different heads specialize on different puzzle types? We analyze winner patterns across the test set for WTA K=4:

Winner distribution. Head 0 wins 31% of puzzles, heads 1–3 win 23%, 24%, 22% respectively. No single head dominates, confirming that diversity is maintained.

Puzzle-head affinity. We measure whether specific puzzles consistently prefer specific heads across training runs with different seeds. Correlation is weak ($r = 0.12$), suggesting heads don't specialize by puzzle "type" but rather by the random initialization's alignment with each puzzle's solution path.

Winner switching during reasoning. For puzzles requiring multiple H-steps, we track which head wins at each step:

H-step	0	1–2	3–4	5+
Same winner as final	62%	78%	89%	97%

Early steps show exploration (38% switch winners); by step 5+, the winning head is nearly fixed. This validates the "explore then exploit" dynamic.

5.12. Difficulty vs. Halting Time

We measure puzzle difficulty using a classical backtracking solver's step count. Figure 3 shows strong correlation ($r = 0.73$) between solver difficulty and neural halting time. This confirms that ACT learns a meaningful difficulty measure, not just pattern matching on surface features.

Notably, the correlation is stronger for WTA-trained models ($r = 0.73$) than baseline ($r = 0.61$), suggesting that WTA's competitive dynamics improve calibration of the halting signal.

5.13. Limitations

- **Sudoku-specific.** We evaluate only on Sudoku-Extreme. Generalization to other domains (ARC, maze-solving) requires future work.
- **Training cost.** WTA training requires $K\times$ forward passes per iteration. For K=4, this is modest, but larger K may be prohibitive.
- **Head collapse.** We observe that heads converge to similar solutions by step 5+. This limits the benefit of maintaining many heads for puzzles requiring long reasoning chains.
- **Bifurcation puzzles.** Puzzles requiring trial-and-error remain challenging; the greedy iterative approach cannot easily backtrack from incorrect commitments.

6. Conclusion

We began with an empirical observation: when ensembling iterative reasoners, selecting by *completion speed* dramatically outperforms probability averaging. This led us to a hypothesis—inference speed is an implicit confidence signal—and a method: winner-take-all training that internalizes ensemble diversity within a single model.

Our results on Sudoku-Extreme are striking: a single 7M parameter model achieves 96.9% $\pm$ 0.6% puzzle accuracy (best seed: 97.6%), matching a 3-model halt-based ensemble and improving 11pp over the TRM baseline. The key insight is not architectural but algorithmic: by letting K=4 parallel hypotheses compete during training, we teach the model to find confident solutions faster.

The broader implication is that **how quickly** a model converges may be as informative as **what** it outputs. This connects to a rich literature on speed-accuracy tradeoffs in human cognition, where fast decisions correlate with confidence. Adaptive computation mechanisms like ACT may be learning to detect this same signal.

Future directions.

- **Beyond Sudoku.** Validating on ARC, maze-solving, and other reasoning benchmarks.
- **Scaling K.** Understanding why K=4 saturates and whether harder domains require more heads.
- **Head specialization.** Encouraging heads to learn diverse strategies rather than converging to similar solutions.
- **Theoretical foundations.** Formalizing the connection between convergence speed and solution quality.

References

Asadulaev, A., Banerjee, R., Karray, F., and Takac, M. Deep improvement supervision. *arXiv preprint arXiv:2511.16886*, 2025. URL https://arxiv.org/abs/2511.16886.

Banino, A., Balaguer, J., and Blundell, C. Pondernet: Learning to ponder. *arXiv preprint arXiv:2107.05407*, 2021.

Child, R. et al. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Cruz, R. M. O., Sabourin, R., and Cavalcanti, G. D. C. Deslib: A dynamic ensemble selection library in Python. *Journal of Machine Learning Research*, 21(8):1–5, 2020.

Dehghani, M. et al. Universal transformers. In *ICLR*, 2019.

Dietterich, T. G. Ensemble methods in machine learning. In *International Workshop on Multiple Classifier Systems*, pp. 1–15. Springer, 2000.

Gomes, C. P. and Selman, B. Algorithm portfolios. In *Artificial Intelligence*, volume 126, pp. 43–62, 2001.

Graves, A. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Grossberg, S. Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science*, 11(1): 23–63, 1987.

Hawkins, J. *A Thousand Brains: A New Theory of Intelligence*. Basic Books, 2021.

Heitz, R. P. The speed-accuracy tradeoff: History, physiology, methodology, and behavior. *Frontiers in Neuroscience*, 8:150, 2014.

Hottung, A., Mahajan, M., and Tierney, K. PolyNet: Learning diverse solution strategies for neural combinatorial optimization. In *ICLR*, 2025. URL https://arxiv.org/abs/2402.14048.

Jolicoeur-Martineau, A. et al. Tiny recursive models. *arXiv preprint arXiv:2510.04871*, 2025. URL https://arxiv.org/abs/2510.04871.

Jordan, K. et al. Muon: An optimizer for hidden layers in neural networks. *arXiv preprint*, 2024. URL https://kellerjordan.github.io/posts/muon/.

Luby, M., Sinclair, A., and Zuckerman, D. Optimal speedup of las vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

Park, S., Kim, S., Na, B., and Yoon, S. T2FSNN: Deep spiking neural networks with time-to-first-spike coding. *arXiv preprint arXiv:2003.11741*, 2020.

Ratcliff, R. and McKoon, G. The diffusion decision model: Theory and data for two-choice decision tasks. *Neural Computation*, 20(4):873–922, 2008.

Rumelhart, D. E. and Zipser, D. Feature discovery by competitive learning. *Cognitive Science*, 9(1):75–112, 1985.

Shazeer, N. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Shazeer, N. et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.

Thorpe, S., Delorme, A., and Van Rullen, R. Spike-based strategies for rapid processing. *Neural Networks*, 14(6-7): 715–725, 2001.

Tolstikhin, I. et al. MLP-Mixer: An all-MLP architecture for vision. In *NeurIPS*, 2021.

Xia, G. and Bouganis, C.-S. Window-based early-exit cascades for uncertainty estimation: When deep ensembles are more efficient than single models. In *ICCV*, pp. 17322–17334, 2023.

A. Approaches That Failed

We document approaches that failed to improve over baseline, as negative results inform future research directions. All experiments below achieved *lower* accuracy than standard TRM training.

A.1. Diversity Through Training Losses

Several approaches attempted to create diverse solution paths during training, motivated by the success of halt-first ensembling at inference.

Forced trajectory divergence. Adding a loss penalizing high cosine similarity between trajectories from different input permutations. Despite successfully reducing similarity (0.83 final), accuracy dropped from 86.3% to 48.2%—the model learned to satisfy both permutations with *worse* solutions rather than better diverse ones.

Inter-H diversity loss. Penalizing consecutive H-iteration similarity to prevent "redundant" iterations. This broke iterative refinement: each step began undoing the previous step's work. Healthy models show $\cos(z_H^{(h)}, z_H^{(h+1)}) > 0.9$; low similarity is harmful.

Multi-head output diversity. Training $K{=}3$ separate value heads on a shared trunk with diversity loss *subtracting* similarity. Heads collapsed to identical outputs within 1000 steps despite the diversity pressure. The shared trunk creates a convergence attractor that overwhelms any loss signal.

A.2. Architectural Diversity Mechanisms

Slot attention. Replacing z_H with $K{=}3$ competing slots, forcing cells to distribute information via softmax competition. Slots collapsed to identical representations—the shared reasoning blocks (MLPMixer) dominate any structural diversity mechanism.

K-specific full-rank weights. Giving each of $K{=}3$ hypotheses completely separate reasoning parameters ($3\times$ model size). Despite no weight sharing, all hypotheses converged to the same solution. Separate weights $\neq$ diversity; training dynamics must force divergence.

Learned z_H initialization. A puzzle-encoder MLP mapping inputs to per-puzzle initial states. Accuracy dropped 1.9%—fixed initialization outperforms learned initialization, which adds noise and overfitting risk.

Multi-init training. $K{=}3$ learned (z_H, z_L) initialization pairs with first-to-halt racing. Initial advantage vanished by step 2500; shared weights caused all initializations to converge to the same solution.

A.3. Loss Weighting and Curriculum

Per-H label smoothing. Softer targets early (exploration), sharper late (commitment). Consistently 10+ percentage points behind baseline—high average smoothing hurt convergence without compensating benefits.

Confidence-weighted loss. Upweighting high-entropy cells. Initially neutral but diverged late (–3% at convergence); extra weighting destabilized training.

Stuck-puzzle loss weighting. $3\times$ weight for puzzles where $q_{\text{halt}} < 0$. With 91% of puzzles marked "stuck" early in training, this overwhelmed the loss landscape.

A.4. Fixed-Point and Contraction Losses

Fixed-point loss. Penalizing $\|z_H^{(h+1)} - z_H^{(h)}\|$ to encourage convergence. Accuracy dropped to $\sim$65%. The model needs continued iteration even when states are similar.

Validity loss. Auxiliary loss encouraging predictions to satisfy Sudoku constraints (no duplicates per row/col/box). All variants (symmetric, low-temperature, Gumbel) failed at $\sim$65% accuracy.

A.5. Input Manipulation

Input corruption. Corrupting 50% of given digits during training (diffusion-inspired). Destroyed learning—the model needs constraint information intact. This contrasts with corrupting *predictions* (z_H noise), which helps marginally.

Stochastic cell selection. Updating random 50% of cells per H-step (ConsFormer-inspired). Failed by –3%: TRM needs all cells to propagate constraints simultaneously.

A.6. Inference-Time Mechanisms at Training

Binary reset during training. Resetting z_H/z_L when $q_{\text{halt}} < 0$. DOA at step 500—destructive gradient signals from 41% of samples stuck at maximum iterations.

Entropy regularization. Encouraging uncertainty at early H-steps. Accuracy dropped 0.5%: the model benefits from committing early on easy cells and propagating, not from hedging.

A.7. Architecture Modifications

Causal attention. Replacing MLPMixer with causal TransformerBlock. Failed at –28% behind baseline: TRM requires *bidirectional* constraint propagation. Causal masking prevents cell 80 from seeing cell 0, breaking Sudoku's global structure.

Larger model. Increasing hidden dimension from 512 to 768. Unstable training with eventual collapse—the MLP-Mixer architecture doesn't benefit from extra capacity on this task.

A.8. Summary

Approach	Δ vs Baseline
Forced trajectory divergence	–38%
Causal attention	–28%
Inter-H diversity loss	–20%
Multi-head diversity loss	–15%
Slot attention	–12%
Per-H label smoothing	–11%
Stochastic cell selection	–3%
Confidence-weighted loss	–3%
K-specific full-rank	–2%
Learned z_H init	–2%
Entropy regularization	–0.5%

Table 3. Summary of failed approaches, sorted by accuracy drop.

The common failure mode: approaches that work for other architectures (diffusion, ViTs, LLMs) often fail for TRM because iterative refinement has different dynamics than single-pass inference or autoregressive generation. TRM needs:

1. Bidirectional information flow (no causal masking)
2. High iteration-to-iteration similarity (incremental refinement)
3. Early commitment on easy cells (propagation, not hedging)
4. Intact input constraints (no corruption)

The success of WTA training, in contrast, works *with* these dynamics: it creates diversity in the *initialization* (where it matters) while allowing each hypothesis to follow normal iterative refinement.